Group 601 Presents:

Game Nite

WITH NEW UPDATES!

TECH STACK

- FRONTEND: REACT + TYPESCRIPT
- BACKEND: NODE.JS + EXPRESS + SOCKET.IO
- DATABASE: MONGODB
- ARCHITECTURE: MODULAR GAME DESIGN

https://github.com/neu-cs4530/spring26-project-group-601-1

PROJECT DESCRIPTION

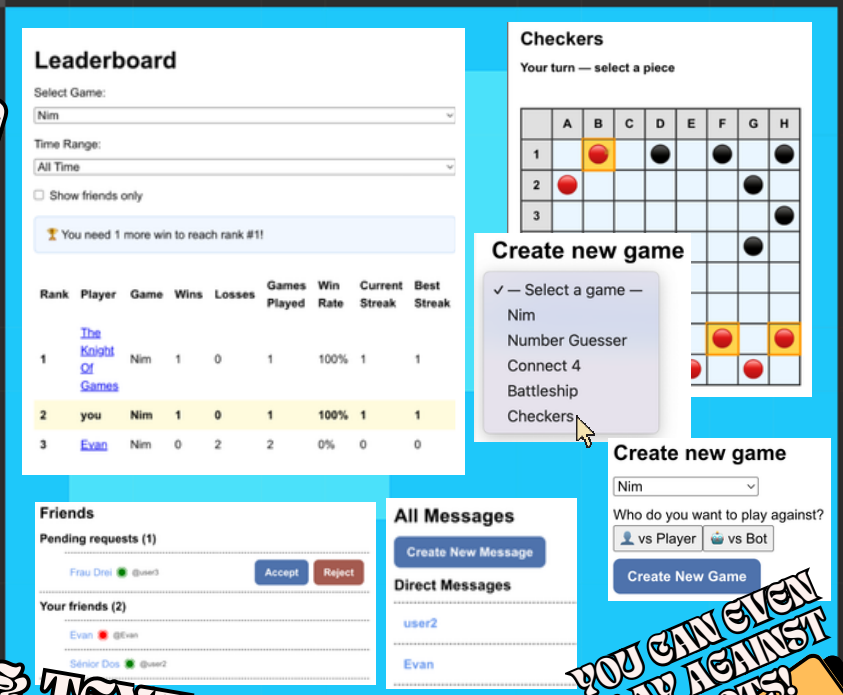
GAMENITE IS A MULTIPLAYER GAMING PLATFORM THAT WE ENHANCED WITH SOCIAL, COMPETITIVE, AND GAMEPLAY FEATURES TO INCREASE ENGAGEMENT. WE INTRODUCED A FRIENDS SYSTEM, LEADERBOARDS, NEW GAMES & AI-POWERED OPPONENTS, ALLOWING USERS TO CONNECT, COMPETE, AND PLAY ANYTIME.

DESIGN DECISIONS

ALL FEATURES ARE BUILT MODULARLY. GAMES, BOTS, AND REAL-TIME UPDATES ARE DECOUPLED SO EACH CAN BE EXTENDED INDEPENDENTLY.

SEE WHERE YOU STAND

PLAY NEW GAMES!



WHAT'S NEXT??

- SINGLE-PLAYER GAMES!
- BETTER UI DESIGN!
- BOT DIFFICULTY VARIATION FOR MORE GAMES!
- MOBILE VERSION!

ADD & TEXT FRIENDS!

YOU CAN EVEN PLAY AGAINST AI BOTS!

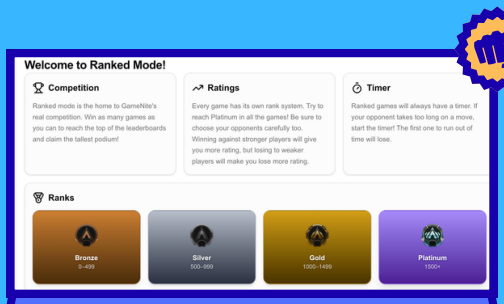
TRY IT OUT NOW:

https://spring26-project-group-601-1.onrender.com/

BROUGHT TO YOU BY ETHAN, EVAN, CHRISTOPHER, NALINI

RankedNite!

By: Ange Li, Hui Wang, Johan Almanzar, Kevin Mo
Frontend Tech Stack: TypeScript + React + Vite w/ Tailwind CSS, Axios
Backend Tech Stack: Express + Node.js w/ Socket.IO, Zod
Testing Tech Stack: Vitest, Playwright



Ranked Mode

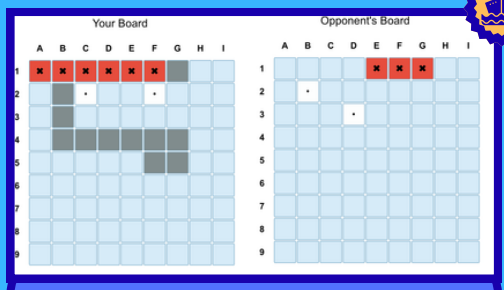
Ranked Mode brings a new competitive playground to BetNite!

- Play against other players to rank up in each game!
- Aim for the top of the leaderboards!
- Utilize the new timer system to keep ranked games fast-paced and exciting!

GitHub Repo: https://github.com/neu-cs4530/spring26-project-group-604
Website: https://spring26-project-group-604.onrender.com/

Why RankedNite?

GameNite had a key problem: its lack of features that encourage replay-ability on the platform. The only key reward in the platform was satisfaction over winning or losing. Our project solved this problem by introducing an exciting new game, a new ranked system, and a leaderboard to keep track of your competition!



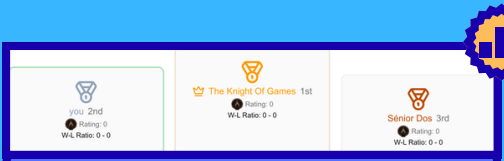
Battleship

Introducing a totally original game: Battleship! (Do not sue us Hasbro)

- Place battleships manually or through random placement!
- Sink your opponent's ships to win the game!
- Choose Grid sizes to play on!

Why build this?

Ranked System is built to be entirely separate from the Casual game mode. Having two separate game modes allows easygoing players to play with their friends for fun, while competitive players can strive to reach their goal rank! Rankings are also separated by game type, so that users can have different ranks for different games. Reaching Platinum in Nim only means you can strive to reach the top rank in Number Guesser and Battleship! Battleship is built from the ground up as an exciting new game that showcases skill expression in Ranked mode. With the ability to place and sink ships, every Battleship match is bound to be unique!



Leaderboard

Track your progress in Ranked mode through the Leaderboard!

- View your competition for any given game!
- See who's on the podium!
- View every player's Win/Loss ratio and Rating!

What's Next?

- Introduce Ranked Seasons! Every few months, reset player ranks so the Ranked game mode always stays competitive!
- Introduce a Friends List! Add friends, block players, and invite friends to join your game lobby!
- Introduce more games! Add games like Chess to bring more fun to RankedNite!
- Introduce chat emotes and reactions! Give players more ways to showcase their personality and talk to their opponent!
- Introduce rewards to top players! Incentivise climbing up the leaderboards even more if players receive special awards or items.

SOCIAL INTERACTIONS & REPORT SYSTEM

Our New Features:

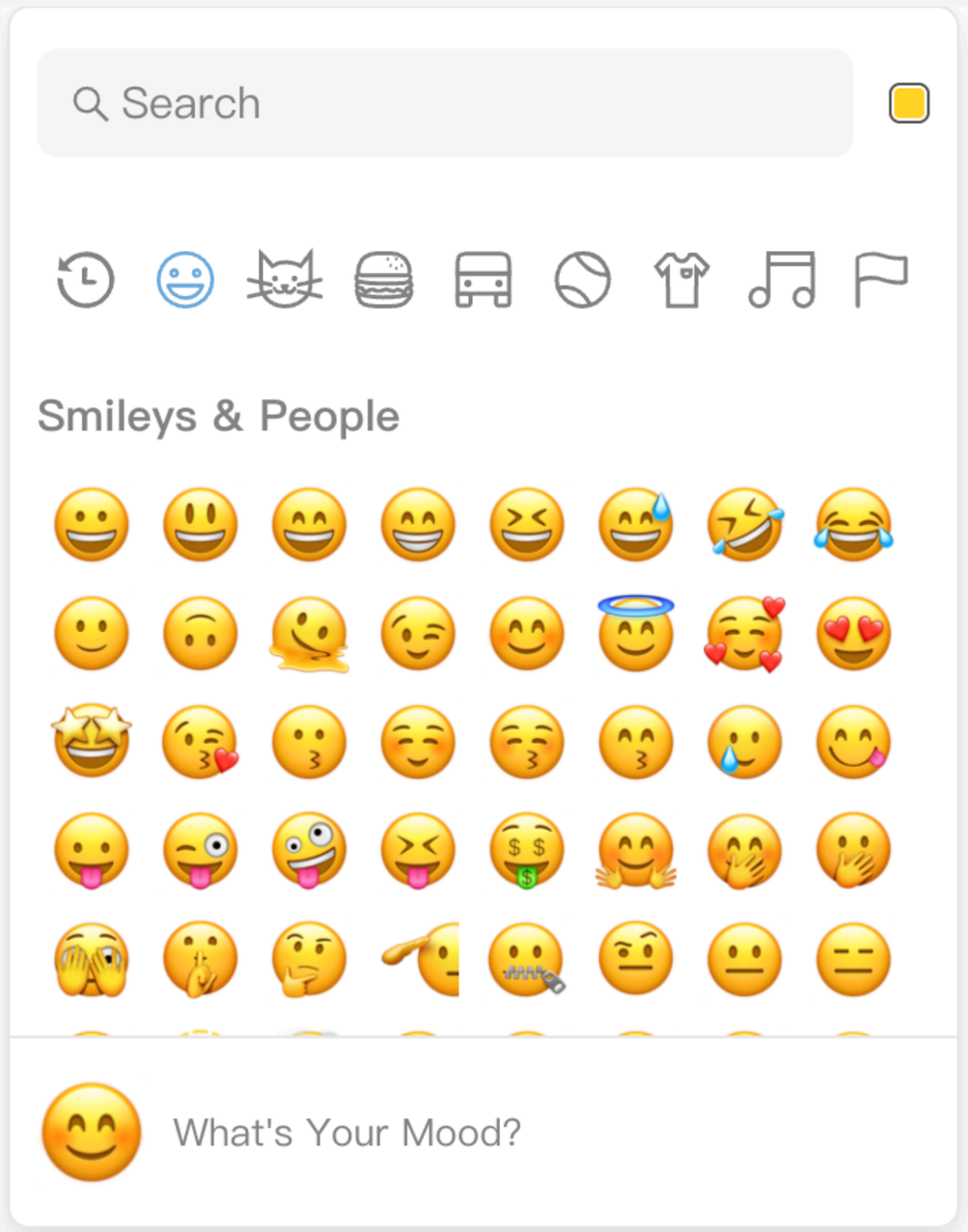
- File Upload and Preview
- Add Emoji and Super Reaction
- Friend Request and Game Invitation

OUR DESIGN:

- Users can try more ways to interact each other
- Users can report others who make them uncomfortable

OUR GOAL:

- GameNite is being developed into a gaming platform with enhanced social features, while the reporting system serves to maintain a healthier environment for the platform.



GROUP 605

Yiyang Zhang
Zihao Ma
Hanwen Zeng
Shoumik Majumdar

https://spring26-project-group-605-1.onrender.com/

https://github.com/neu-cs4530/fall25-project-fall25-project-group-516

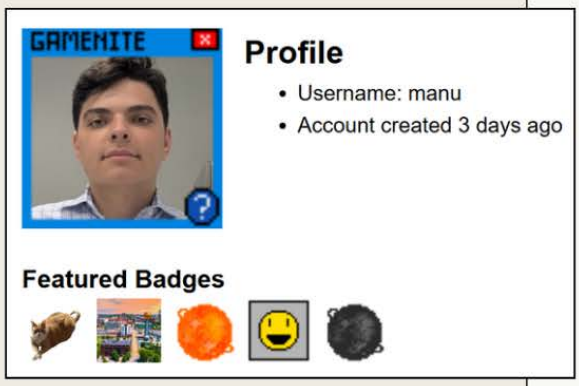
NEWEST GAMENITE FEATURES

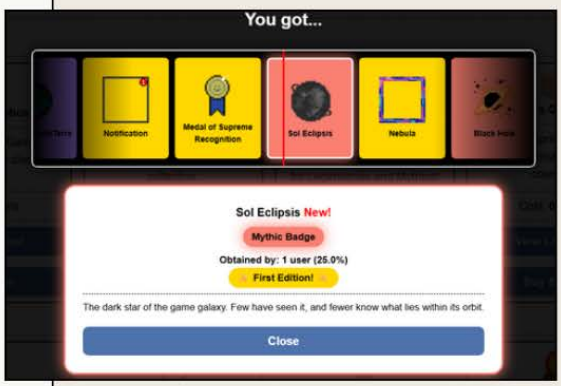
GAMBLING EDITION

Team Manu:
Ari Prakash, Joshua Goldberg,
May Du, and Saul Manzanares

Identifiable Profiles

- Profile Pictures
  - Works anywhere!!!!
- Custom biographies
- Achievements
- Displayable Items
  - Profile frames
  - Badges





Gambling Gambling!!

- Specific game betting
  - Publicly bet on games for Luners
  - Privately bet or trade items
- View Odds
- Bet on multiple games at once
- Gameboxes

What's Next???

- More profile customization
  - Banners?
- Multiple player trade
- Private Luner betting on games
- EVEN MORE GAMEBOXES

Links:

Website:

- https://spring26-project-group-606.onrender.com

Repo:

- https://github.com/neu-cs4530/spring26-project-group-606

GAMENITE

Developed by: Team 607 - Scott Brinkley, Joey Yun, Shree Singhal
GitHub Repo: https://github.com/neu-cs4530/spring26-project-group-607-1

PROJECT SUMMARY

The goal was to extend matchmaking features in GameNIte. This included support for friend requests, pinging other users and inviting friends to games, and the ability to play with an AI player.

TECH STACK

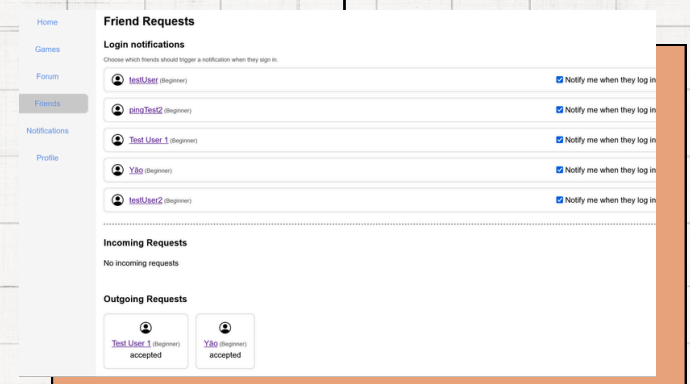
This project was developed using TypeScript with Node.js on the backend and React on the frontend. Our application uses MongoDB as the database.

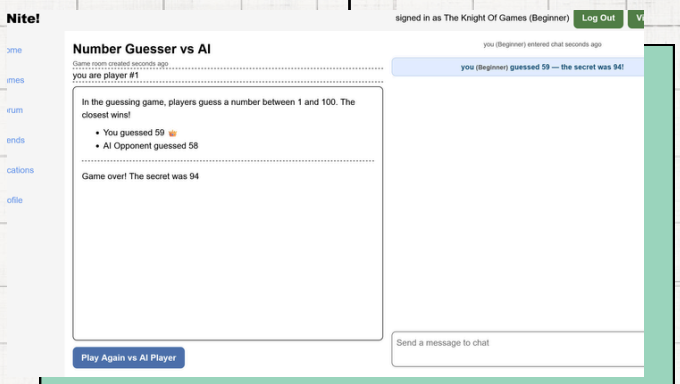
DESIGN DECISIONS

We used a frontend architecture which emphasized reusable components and hooks to avoid duplication and improve readability. On the backend, we maintained a clear separation of concerns with a distinct service layer. We used websocket functions on the backend when needing live data updates, avoiding unnecessary calls to the API.

WHAT'S NEXT?

GameNite can be extended to support messaging between friends and the ability to block other users for security. Additionally, the AI player can be better integrated into the application, perhaps with a more advanced model to create moves.





SPRING26-PROJECT-GROUP-607-1.ONRENDER.COM/

GameNite

Create, Share, and Compete with Custom Word Puzzles

We enhanced **GameNite** by adding **Connections** — a word puzzle game where players group 16 words into 4 hidden categories — along with tools to **create custom puzzles** and **compete on leaderboards**!

Team 608: Brian Gerber · Emma Mathew · George Mattis · Jude Riley

New

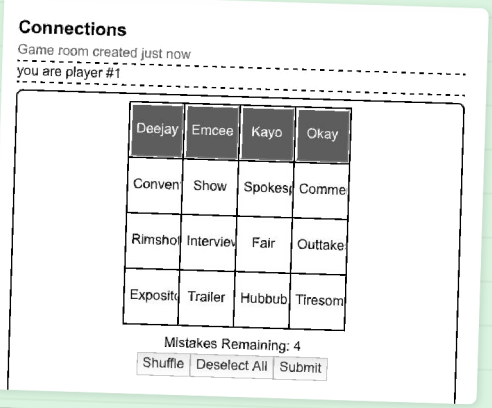
Features!

- Connections Game
- Puzzle Creator
- Leaderboards
- Puzzle Browser
- Player Stats

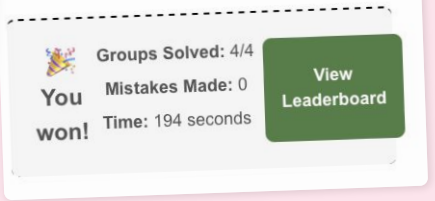
Playing Connections

Group 16 words into 4 hidden categories — can you figure them out? Get feedback on your guesses *(4 mistakes allowed!)*. See your stats when you finish: mistakes made, groups solved, and completion time. Play hundreds of community-created puzzles, or the default board!

The Game Board



Your Results



Yay!

Creating & Sharing Puzzles

- Create custom Connections puzzles
- Browse community puzzles with scrambled word previews
- Search by title, sort by date or creator
- Your creations appear on your profile for others to play

Browse Puzzles

Hundreds of puzzles!

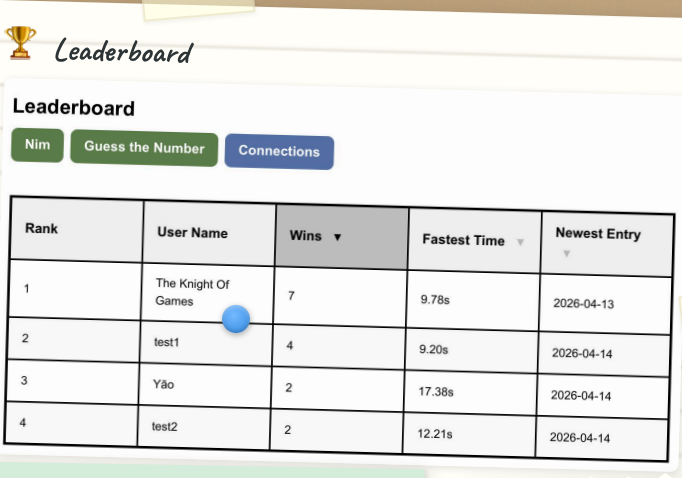
Play community-created puzzles or make your own!

Your puzzle, your rules

Competing on Leaderboards

- Leaderboards for every game (Nim, Number Guesser, Connections)
- Track total wins & fastest completion time
- Sort by wins, speed, or recent activity
- Auto-updates when you finish the game!

Leaderboard



Someone ahead?

Everyone's stats are tracked!

Beat their time!!!

Technology

React · TypeScript · Node.js · MongoDB · Socket.io

Try it live!

spring26-project-group-608.onrender.com

Source Code

github.com/neu-cs4530/spring26-project-group-608

What's Next?

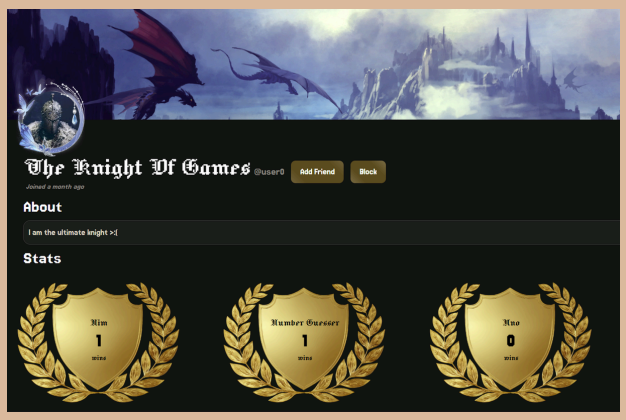
- → Multiplayer racing mode
- → Daily curated puzzles with difficulty ratings
- → Puzzle rating & favorites system
- → More detailed analytics & player stats

Group 609

GameNite

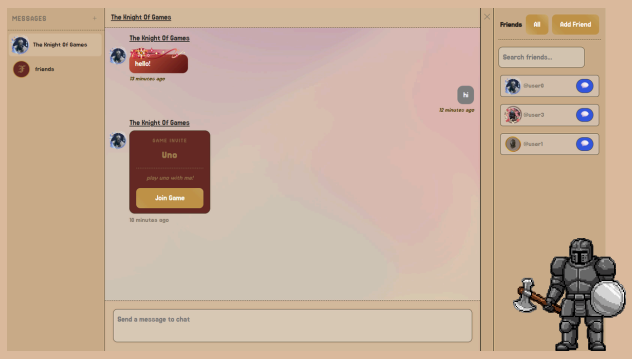
Justin Chen, Rebecca Lee, Amy Lu, Aaron Xu

GameNite functions more as a collection of isolated gaming experiences rather than a cohesive community. There is a lack of social infrastructure, limited games, and a generic interface that prevents users from expressing themselves, resulting in low retention. To resolve this, our project introduces a comprehensive social infrastructure that allows for friending, direct messaging, and inviting friends to games with capabilities to express yourself through profile pictures, personal bios, and personalization.



Tech Stack & Design Decisions

GameNite is built with a React/TypeScript frontend and a Node.js/Express backend, connected by Socket.io for real-time updates (game moves, chat, presence) and REST for standard CRUD operations. Game state is kept server-authoritative, with MongoDB (or in-memory storage) for persistence. Shared Zod schemas across client and server keep the API contract consistent. Visually, we wanted to display a medieval aesthetic with the familiar layout and features of modern social apps like Discord and Tumblr.



What's next?

Future development will focus on adding more games and enriching the forum with post-locking, downvoting, and upvoting mechanics. We plan to unify the platform's visual identity while enhancing the storefront with sorting options, including ascending and descending price filters.

Repo: https://github.com/neu-cs4530/spring26-project-spring26-project-group-609

Deployed Site: https://gamenite-609.onrender.com/

CasinoNite

New Features

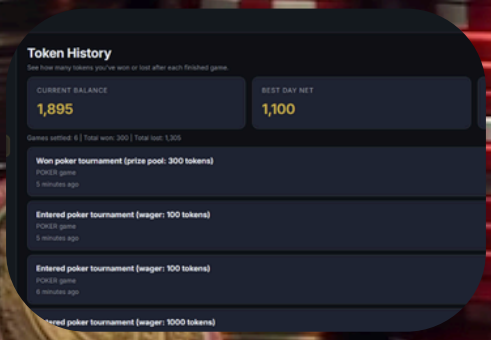
- Friends system with send, accept, decline, cancel, and remove flows
- Server-side token balance with daily refill logic
- Token balance displayed directly in the user interface
- Token wagering integrated into multiplayer game flows
- Poker with server-authoritative hidden information
- Automated tests, CI, and public deployment

Design Decisions

- Server-authoritative game state: all game logic runs on the server. Clients receive read-only views, preventing cheating
- Player-specific socket events: private information (i.e. hole cards in Poker) is sent only to the owning player's socket, never broadcast to all
- Token wager coverage enforcement: server checks available balance at both game creation and join time
- Shared type package: a single @gamenite/shared module defines types used by both client and server, eliminating runtime type mismatches

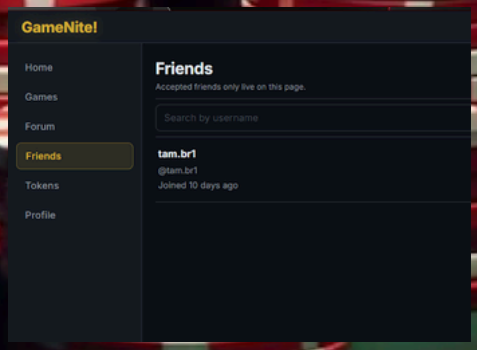
Future Work

- Expand poker with more complete betting rules, side pots, and all-in support
- Add more casino-style games that use the token economy
- Improve friend features with online status and better notifications
- Add richer token history, analytics, and player progression stats
- Polish the UI and make the overall experience feel more cohesive
- Strengthen reconnect handling and multiplayer reliability



TOKENS!

FRIENDS!



POKER!

Tech Stack

- Frontend: React + TypeScript, React Router, Socket.IO client
- Backend: Node.js + Express, Socket.IO server, Deno-compatible TypeScript
- Database: Firebase Realtime Database for persistent game, user, and token state
- Deployment: Render (server) + Vite (client build)

Group 610

Benjamin, Atif, Krish, Brandon

Website: CasinoNite

Github Repo: Repo

GameNite

Social & Privacy Feature - CS4530 Spring 2026 - Group 611

Varsha Basker - Evan Yang - Heng Xu - Sunny Shen

Demo: https://spring26-project-spring26-project-group-k1ml.onrender.com/
Repo: https://github.com/neu-cs4530/spring26-project-spring26-project-group-611

Problem

GameNite originally treated all players and games as fully public, which made users feel anonymous and limited their ability to build meaningful connections. There was little control over who you could interact with or play with, which created a less personalized experience.

Our Solution

Friends & DMs

Send requests, accept/decline, message friends privately

Private Games

Hosts invite friends, control visibility, manage players

Profile Stats & Personal Info

Win/loss streaks, game history & Hometown, age, favorites with visibility controls - Public / Friends / Private

Tech Stack

Frontend	React + TypeScript
Backend	REST API
Auth	Session-based access control
CI/CD	GitHub + Render
Testing	ViTest unit + PlayWright E2E tests
Database	MongoDB

What's Next

- **Host ending the game** for all players
- **User blocking** and moderation tools
- **Group chats** for game lobbies
- **Leaderboards** with friend-aware rankings
- Refactor access control into reusable middleware

3 User Stories

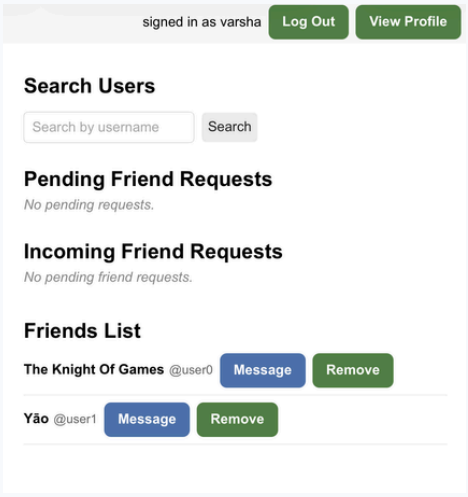
30+ Tasks Done

4 Sprints

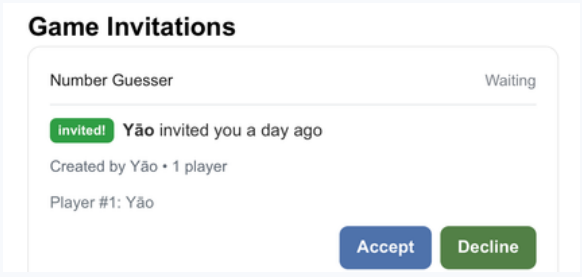
Feature Screenshots

Home - Public & Private Games

Friends - Search & Requests



Game Invites - Accept / Decline



Profile Stats

Friends System

Search for players, send/accept requests, manage your network. Friends unlock DMs, profile stats, and private game access.

Private Games

Hosts set public or private visibility at creation. Invite-only games let hosts manage players and kick participants.

Profile & Stats

Track games played, win/loss streaks, favorite game. Each stat configurable: Public, Friends-only, or Private.

Direct Messaging

Friends message each other privately outside game rooms, alongside the existing public forum.

Design Decisions

- **Friend-gate invites**

Only friends can be invited, enforced server-side

- **Role-based access**

Host vs player permissions inside game rooms

- **Visibility middleware**

canViewStats() centralizes all access control

- **Prioritized sprints**

Stories worked in parallel with friends system given highest priority

CS4530 FINAL PROJECT: PARTY ROOM

A Host Led Party Game Platform with Mini Games

Authors

Corina Torres, Quinn Louie, Nick Connors, and Daniel Karma

Emails

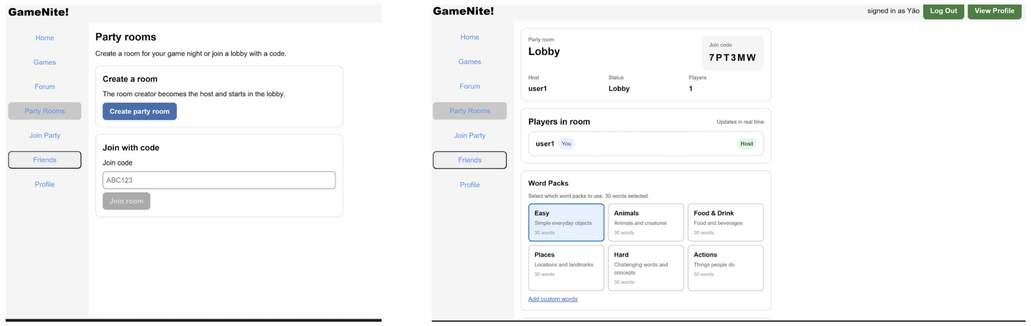
torres.c@northeastern.edu | louie.q@northeastern.edu | connors.ni@northeastern.edu | karma.d@northeastern.edu

Project Overview

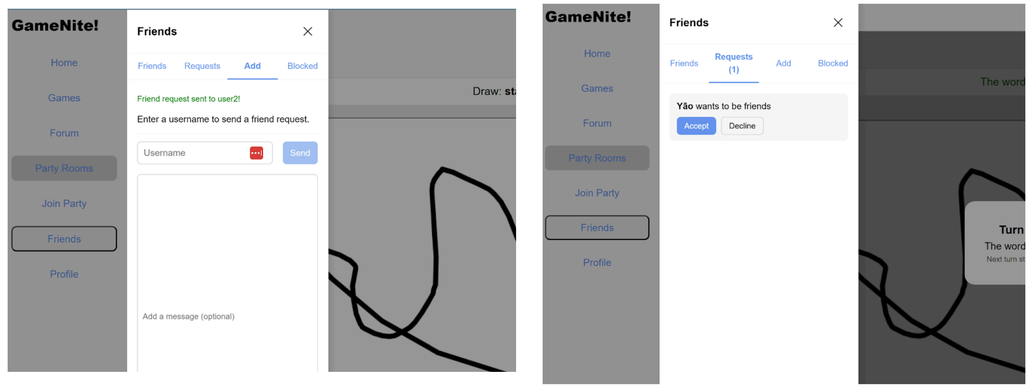
GameNite currently supports games, but it does not support a “party mode” - a room where multiple players can play multiple mini games together with a winner at the end. Without this dedicated feature, players must coordinate outside of GameNite, queue up into multiple separate game rooms, and then tally on their own who won their “party” session. It’s a much more isolated experience, with little social interaction between players. GameNite also does not have a friend system in place for players to add and play with each other.

Our project introduces a Party Room that adds a new kind of game experience: host-led, multi-round, real-time party games. A host can create a room, invite players through code or directly through GameNight as a friend, and start a session. Players join and participate in various mini games with the winner of each receiving a score, and a final winner decided at the end. Our project also introduces a small friend system, where players can add one another so that they can play games together, and invite each other directly through GameNite. We also implemented 2 new mini games.

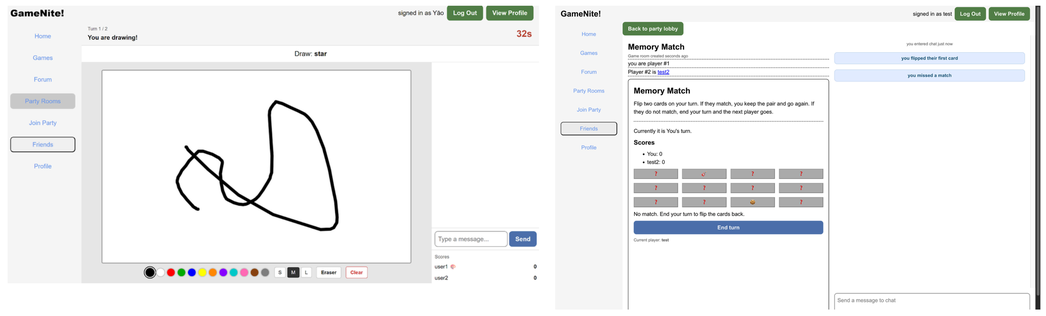
GameNite is a web application for asynchronous games and community features. Our team extended it with a party room experience: players gather in a shared lobby, vote on the next mini-game, and play in real time over WebSockets. Party rooms support multiple game types (Nim, Number guesser, Memory Match, and Skribbl), a scoreboard across rounds, and integration with the friends system (sending requests from the lobby, inviting friends into a room, and managing friends through the sidebar).



Creating a Party room, and the lobby where the host can choose word packs for Skribbl



Friend system UI - set up as a side bar over the main page. You can request friends with a message, accept, decline, and block



Skribbl and Memory Match games implemented as shown above

Technology Stack & Design

Our project extends the existing GameNite web application by introducing a real-time party room system built on a full-stack TypeScript architecture with a clear separation between client, server, and shared types. The frontend uses React with component-based UI and custom hooks for managing state and socket.IO communication, while the backend leverages Node.js with a service and repository layer to handle party room logic such as room creation, voting, and score tracking. Real-time synchronization is achieved through socket.IO events, enabling live updates for player lists, votes, and game state across all clients. A key design decision was to reuse and extend the existing game framework and shared type system to maintain consistency, while adopting a centralized server-authoritative state model to ensure reliable synchronization and simplify multi-user interactions.

Future Work

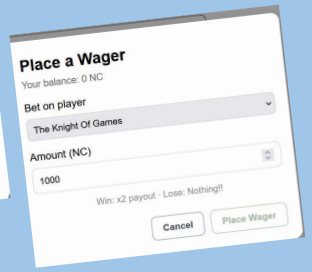
Future work could focus on expanding both functionality and system design. One direction is adding new features such as spectator mode, host transfer capabilities, or additional mini-games to further enhance the party experience. These were all features we had described in our conditions of satisfaction as extensions, so these would be “next up” to implement. Improvements could also be made to the architecture, such as refining abstractions between components and creating more modular interfaces for game integration, making it easier to add new game types without modifying core logic.

GitHub Repo and Demo Site

https://github.com/neu-cs4530/spring26-project-group-612

https://spring26-project-group-612-46ls.onrender.com/

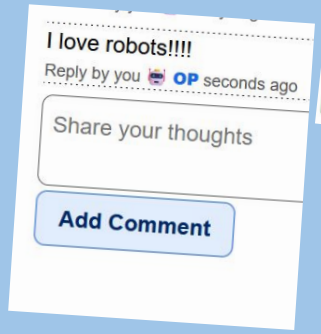
Group 613: Entertainment



Use currency to show your support or risk it all!

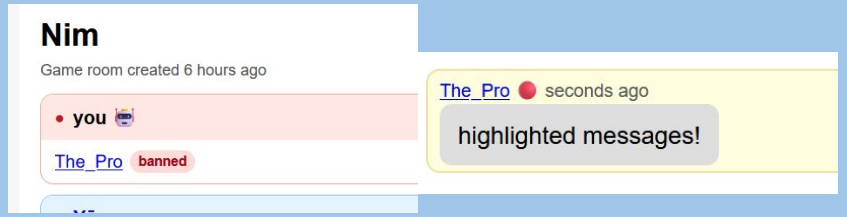
Core Design

Rather than a gamenite for players with spectators, why not have a gamenite for entertainers with supporters?

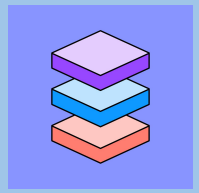


Express Yourself!

Ban or Highlight spectators!



- Front End–React
- Backend–TypeScript
- Database–MongoDB



What's Next:

- DMs!
- Recommended games!
- Rivalry Systems!
- More detailed wagers
- And more!!!!

Website: https://spring26-project-group-613.onrender.com/

Repo: https://github.com/neu-cs4530/spring26-project-group-613

By Lukas Tegge, Ben Jaccaci, Smaran Teja, and Brendan Friday